

Capitolo 18 — PROC-002 — Workspace Pre-Sync

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-18
Data master	2026-08-29
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/18_proc_002_workspace_pre_sync.md
Source SHA	360bc14
Release	2026-08-31T20:48:50.990Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

Parte VI — Il Libro dei Processi WCM Stato: FROZEN **Data:** 2026-08-29 **Scope:** WCM generale, domain-agnostic

18.0 Prima di lavorare bisogna sapere su quale realtà stiamo lavorando

Nel capitolo precedente abbiamo visto come WCM governa il ciclo di vita di un Service Job. Un lavoro può essere definito, autorizzato, preso in carico e infine chiuso soltanto quando il suo stato è persistente e verificabile.

Ma appena quel lavoro deve operare su un repository Git compare un problema precedente a quasi tutti gli altri:

Il workspace locale che l'esecutore sta guardando rappresenta davvero lo stato remoto corrente su cui dovrebbe lavorare?

In un sistema distribuito esistono almeno due realtà contemporanee:

```
REPOSITORY REMOTO
+
WORKSPACE LOCALE
```

Possono coincidere. Possono essere semplicemente arretrate di qualche commit. Possono divergere. Possono contenere modifiche locali non ancora comprese. Possono trovarsi sul branch sbagliato. E soprattutto possono **sembrare coerenti pur non essendolo**.

PROC-002 esiste per impedire che un agente, un servizio o un'automazione comincino a ragionare e a modificare file assumendo come vera una fotografia locale non verificata.

Il principio centrale è:

```
PRIMA DI USARE IL WORKSPACE COME REALTÀ OPERATIVA,
SINCRONIZZALO E VERIFICALO.
```

18.1 Che cos'è il Workspace Pre-Sync

Il **Workspace Pre-Sync** è il processo con cui WCM verifica e riallinea in modo sicuro un workspace Git autorizzato prima di eseguire lavoro che dipende dallo stato corrente del repository remoto.

La sua funzione non è "fare Git" in senso generico. Non serve a gestire qualunque strategia di branching, a risolvere conflitti arbitrari o a ripulire automaticamente un repository problematico.

Serve a rispondere a una domanda precisa:

```
POSSO CONSIDERARE  
QUESTO WORKSPACE LOCALE  
UNA BASE AFFIDABILE  
PER IL LAVORO CHE STA PER PARTIRE?
```

Il processo è quindi un **gate di affidabilità pre-esecuzione**.

18.2 Perché "ho già il repository sul disco" non basta

Un workspace locale può contenere una copia perfettamente valida del repository e tuttavia non rappresentare lo stato remoto corrente. Git è distribuito: una copia locale possiede la propria storia, i propri riferimenti e il proprio working tree, mentre il repository remoto può evolvere indipendentemente.

Per esempio:

```
REMOTE MAIN  
A → B → C → D  
  
LOCAL MAIN  
A → B → C
```

Il workspace locale non è corrotto. È semplicemente indietro.

Se l'esecutore comincia a leggere file, valutare configurazioni o produrre modifiche partendo da **C**, mentre la baseline remota è già **D**, può prendere decisioni corrette rispetto a una realtà che non esiste più.

Il problema non è soltanto tecnico. È epistemico:

stiamo costruendo una decisione sulla fonte giusta, ma su una versione non verificata della fonte.

18.3 origin/main non è una prova sufficiente

Uno dei punti più importanti di PROC-002 riguarda il significato di riferimenti locali come:

```
origin/main
```

È facile interpretarli mentalmente come "questo è lo stato attuale di **main** sul remoto". Non è necessariamente vero.

origin/main nel repository locale rappresenta l'ultima conoscenza locale disponibile del branch remoto. Se non è stato eseguito con successo un nuovo:

```
git fetch origin
```

quel riferimento può essere stale.

Quindi:

```
LOCAL origin/main
≠
PROVA DELLO STATO REMOTO CORRENTE
```

prima di un fetch riuscito.

Questa distinzione impedisce di trasformare una cache locale in una falsa source of truth.

18.4 Il trigger

PROC-002 viene attivato:

prima di qualsiasi esecuzione che dipenda dallo stato corrente di un repository Git remoto.

La parola importante è *dipenda*.

Se l'attività deve leggere la baseline corrente, modificare file, confrontare una versione locale con quella remota, eseguire test sulla versione operativa, produrre un commit o riprendere un Service Job che assume una certa baseline, allora la fiducia nel workspace è una preconditione del lavoro successivo.

Non è un rito da eseguire in modo indiscriminato. È necessario quando **la correttezza del lavoro dipende dalla correttezza della fotografia locale**.

18.5 La sequenza canonica

La baseline di PROC-002 definisce questa sequenza:

```
VERIFICA WORKSPACE
↓
VERIFICA BRANCH
↓
VERIFICA WORKING TREE
↓
GIT FETCH ORIGIN
↓
GIT PULL --FF-ONLY
↓
VERIFICA STATO FINALE
↓
WORKSPACE LOCALE AFFIDABILE
```

La sequenza è breve, ma ogni passaggio protegge una classe diversa di errore.

18.6 Primo controllo: il workspace è quello autorizzato?

Prima ancora di chiedere quale branch sia attivo, WCM deve sapere **dove** sta lavorando. Un esecutore può avere accesso a più repository, copie di test, clone storici o workspace temporanei. Per questo il primo controllo è:

```
WORKING DIRECTORY
=
WORKSPACE AUTORIZZATO?
```

Se la risposta non è verificabile, proseguire significherebbe rischiare di applicare comandi corretti al repository sbagliato.

PROC-002 non autorizza a scegliere autonomamente un altro workspace soltanto perché "sembra quello giusto". Il workspace fa parte del perimetro operativo.

18.7 Secondo controllo: il branch è quello previsto?

Una volta verificato il repository, bisogna verificare il branch.

La domanda non è quale branch sia più comodo. È:

```
BRANCH CORRENTE
=
BRANCH AUTORIZZATO DAL CONTRATTO?
```

Se il Service Job o la baseline operativa prevedono **main**, trovarsi su un branch temporaneo cambia il significato dei file osservati. Viceversa, se un lavoro è stato esplicitamente isolato su una branch temporanea, spostarsi autonomamente su **main** può distruggere la separazione prevista.

La regola è quindi:

```
BRANCH MISMATCH
≠
PERMESSO DI CAMBIARE BRANCH
```

Il cambio deve essere compatibile con l'autorità già esistente.

18.8 PROC-002 e PROT-006

PROC-002 si collega naturalmente a **PROT-006 – Branch Ownership & Baseline Sync**.

La baseline corrente del WCM adotta **main** come trunk operativo condiviso. Le branch esistono quando serve isolamento temporaneo e devono avere una exit condition.

Il pre-sync non ridefinisce questa topologia: la verifica.

```
PROT-006  
DEFINISCE LA DISCIPLINA DI BRANCH
```

```
PROC-002  
VERIFICA CHE IL WORKSPACE  
SIA ALLINEATO A QUELLA DISCIPLINA  
PRIMA DELL'ESECUZIONE
```

Sono due responsabilità diverse e complementari.

18.9 Terzo controllo: il working tree è sicuro?

Prima di sincronizzare con il remoto, WCM deve osservare il working tree. Qui entra direttamente:

```
PROT-001 – Git & Working Tree Safety
```

La regola di base è:

una modifica locale non ancora compresa non deve essere cancellata, sovrascritta o aggirata.

Un working tree può essere pulito oppure presentare modifiche. Ma una semplice indicazione di file modificato non dice ancora che cosa significhi quella modifica.

Per questo PROT-001 richiede verifica e classificazione, non reazione automatica.

18.10 Dirty non significa automaticamente "bloccato"

Un working tree modificato è un segnale, non una diagnosi.

Una differenza può rappresentare lavoro reale non committato, un cambiamento intenzionale in corso, un artefatto generato, normalizzazione di line ending, metadata o una modifica estranea allo scope del job.

La baseline quindi non dice:

```
M → BLOCKED
```

Dice, in sostanza:

```
WORKING TREE MODIFICATO  
↓  
COMPRENDI IL DIFF  
↓  
CLASSIFICA  
↓  
DECIDI SE È SICURO PROSEGUIRE
```

La lezione è importante:

il determinismo non consiste nel reagire sempre allo stesso simbolo; consiste nell'applicare sempre lo stesso contratto di verifica allo stesso tipo di evidenza.

18.11 Perché il pre-sync non può cancellare ciò che trova

Supponiamo che il workspace contenga una modifica locale non attesa. La soluzione più veloce potrebbe sembrare:

```
git reset --hard
```

Ma sarebbe una violazione del modello di sicurezza.

Il pre-sync non possiede automaticamente authority per decidere che quella modifica non vale nulla. Per questo la baseline vieta di usare come scorciatoie `reset --hard`, force push, rebase correttivi o merge correttivi non autorizzati.

La regola è:

```
SYNC  
≠  
DISTRUZIONE DELLO STATO LOCALE
```

Il processo deve ottenere un workspace affidabile **senza inventarsi il diritto di cancellare evidenza o lavoro esistente**.

18.12 Il fetch: aggiornare ciò che sappiamo del remoto

Superati i controlli iniziali, il passo successivo è:

```
git fetch origin
```

Il fetch aggiorna la conoscenza locale dei riferimenti remoti senza integrare automaticamente quei commit nel branch locale.

Concettualmente:

```
PRIMA DEL FETCH  
LOCAL REMOTE-TRACKING VIEW  
può essere stale  
  
DOPO FETCH RIUSCITO  
LOCAL REMOTE-TRACKING VIEW  
riflette il remoto osservato in quel momento
```

Questo passaggio separa due domande:

1. che cosa esiste sul remoto?
2. come deve essere aggiornato il branch locale?

È una separazione utile perché evita di confondere discovery e mutazione.

18.13 Fetch riuscito non significa ancora workspace sincronizzato

Dopo il fetch possiamo sapere che il remoto è avanzato, ma il branch locale può essere ancora indietro.

```
origin/main: A → B → C → D
main locale: A → B → C
```

La conoscenza del remoto è aggiornata, ma il filesystem locale continua a rappresentare **C**. Quindi:

```
FETCH PASS
≠
PRE-SYNC COMPLETE
```

Serve il passaggio successivo.

18.14 Perché **git pull --ff-only**

PROC-002 prescrive:

```
git pull --ff-only
```

La parte decisiva è **--ff-only**.

Il pre-sync deve allineare il workspace a una baseline compatibile. Non deve inventare una strategia di integrazione quando la storia locale e remota non consentono un semplice fast-forward.

```
LOCAL
A → B → C

REMOTE
A → B → C → D

RISULTATO
A → B → C → D
```

Non viene creato un nuovo merge commit e non viene riscritta la storia: il branch locale avanza alla baseline remota.

18.15 Il significato del fallimento di **--ff-only**

Se **git pull --ff-only** fallisce, il messaggio operativo non è "proviamo un merge".

Il significato è:

| la sincronizzazione semplice e autorizzata non è possibile nello stato attuale.

Questo può indicare divergenza della storia o un'altra condizione che richiede classificazione.

PROC-002 non autorizza a trasformare automaticamente quel fallimento in merge, rebase, reset o force push. Il fallimento diventa un segnale da gestire entro gli esiti previsti dal processo.

18.16 La verifica finale

Dopo fetch e pull non basta presumere che tutto sia corretto. Il processo richiede una nuova verifica di branch e working tree.

Perché un processo deterministico non considera riuscita un'azione soltanto perché il comando non ha restituito errore. Verifica anche lo stato risultante.

```
OPERAZIONE
↓
POST-CONDITION CHECK
↓
STATO ATTESO?
```

Solo a quel punto il workspace può essere considerato affidabile per il lavoro successivo.

18.17 Il vero output di PROC-002

L'output di PROC-002 non è:

```
"ho eseguito git pull"
```

È:

```
STATO LOCALE AFFIDABILE
PER IL LAVORO AUTORIZZATO
```

Questa distinzione è coerente con una regola più generale del WCM:

| un processo non coincide con l'esecuzione dei suoi comandi; coincide con il raggiungimento verificato della propria post-condizione.

Il comando è un mezzo. La post-condizione è ciò che permette al processo successivo di fidarsi del risultato.

18.18 I tre esiti canonici

PROC-002 prevede tre esiti:

```
PASS
BLOCKED_LOCAL
BLOCKED_WISE
```


Queste categorie impediscono di comprimere qualunque problema in un generico **ERROR**. Ogni esito comunica qualcosa sul prossimo passo possibile.

18.19 PASS

PASS significa che il workspace è stato verificato e sincronizzato in modo compatibile con il processo.

In termini pratici:

- repository corretto;
- branch corretto;
- working tree compatibile;
- fetch riuscito;
- pull fast-forward riuscito o nessun avanzamento necessario;
- stato finale verificato.

A quel punto il lavoro che dipende dalla baseline corrente può procedere.

```
PROC-002 PASS
→
LOCAL STATE TRUSTED
→
NEXT EXECUTION STEP ELIGIBLE
```

PASS non attribuisce authority aggiuntiva: dichiara soltanto soddisfatta la precondizione di affidabilità del workspace.

18.20 BLOCKED_LOCAL

BLOCKED_LOCAL rappresenta un problema tecnico temporaneo risolvibile entro l'authority disponibile.

Non equivale automaticamente a una richiesta di intervento umano. Il significato è:

```
IL PRE-SYNC NON È ANCORA PASSATO,
MA LA RISOLUZIONE RESTA
NEL PERIMETRO OPERATIVO AUTORIZZATO.
```

PROC-002 non inventa una soluzione universale per ogni problema Git: classifica l'esito e impedisce che un workspace non verificato venga trattato come affidabile.

18.21 BLOCKED_WISE

BLOCKED_WISE indica che proseguire richiede qualcosa che il servizio non può legittimamente decidere da solo.

Può emergere, per esempio, una divergenza che richiede una scelta semantica, un branch mismatch che non è autorizzato correggere, una modifica locale di cui non è possibile determinare ownership o destino, oppure la necessità di ampliare scope o authority.

La regola è:

```
MANCANZA DI AUTHORITY
→
NON IMPROVVISARE
```

Il pre-sync deve fermarsi nel punto corretto invece di trasformare una questione di governance in una decisione tecnica automatica.

18.22 Il Pre-Sync come barriera contro lo stale context

Nei capitoli sulla conoscenza abbiamo visto che WCM usa INDEX-FIRST, source precedence e Stop When Sufficient per ridurre il rischio di lavorare su contesto sbagliato.

PROC-002 affronta una versione più concreta dello stesso problema. Un file locale può essere semanticamente corretto e tuttavia essere stale.

```
KNOWLEDGE NAVIGATION
PROTEGGE DA FONTI IRRILEVANTI / NON AUTOREVOLI

WORKSPACE PRE-SYNC
PROTEGGE DA UNA COPIA LOCALE NON VERIFICATA
DELLA FONTE OPERATIVA
```

Sono due livelli diversi della stessa disciplina: **non usare ciò che hai davanti soltanto perché è disponibile.**

18.23 PROC-001 → PROC-002

Nel capitolo 17 abbiamo visto il Pre-Execution Gate del Service Job. Fra gli elementi da verificare compariva il workspace.

PROC-002 rende concreta quella verifica.

```
PROC-001
SERVICE JOB READY / CLAIMED
↓
PRE-EXECUTION GATE
↓
WORKSPACE AFFIDABILE?
↓
PROC-002
↓
PASS
↓
ESECUZIONE PUÒ CONTINUARE
```

PROC-002 non sostituisce PROC-001. È un processo specializzato che fornisce una delle precondizioni necessarie all'esecuzione sicura del job.

18.24 PROC-002 e PROT-001: processo e guardrail

PROC-002 dice **che cosa deve accadere per ottenere un workspace sincronizzato e affidabile**.

PROT-001 impone **come devono essere trattate in sicurezza le operazioni Git e le modifiche locali incontrate lungo il percorso**.

```
PROC-002
=
PERCORSO OPERATIVO DI PRE-SYNC

PROT-001
=
VINCOLI DI SICUREZZA GIT / WORKING TREE
```

Il processo senza il protocollo rischierebbe di diventare aggressivo. Il protocollo senza il processo non definirebbe quando e perché ottenere la post-condizione di sync.

18.25 PROC-002 e PROT-006: sincronizzare senza creare una seconda baseline

PROT-006 aggiunge una dimensione architetturale. La baseline corrente usa **main** come trunk operativo condiviso e tratta le branch temporanee come strumenti di isolamento con exit condition.

PROC-002 deve quindi evitare due errori opposti:

1. IGNORARE IL TRUNK CORRENTE
2. TRATTARE UNA BRANCH TEMPORANEA COME BASELINE AUTONOMA

Il pre-sync verifica la relazione fra workspace, branch autorizzato e baseline remota senza ridefinire l'ownership delle branch.

18.26 Perché non basta leggere direttamente il remoto ogni volta

Si potrebbe pensare che, se il problema è la freschezza del workspace, basti lavorare sempre direttamente sul remoto.

In alcuni casi una capability remota diretta è sufficiente e preferibile. Ma esistono lavori che richiedono un workspace locale o equivalente: build, test, trasformazioni su più file, toolchain locali, render, analisi che dipendono dal filesystem e operazioni Git strutturate.

PROC-002 non afferma quindi che ogni lavoro debba essere locale. Dice:

quando il lavoro dipende da un workspace Git locale, quel workspace deve essere reso affidabile prima dell'uso.

La scelta fra capability diretta e locale appartiene ad altri livelli di routing.

18.27 Esempio astratto: il repository è indietro

Immaginiamo un job autorizzato sul branch **main**. Il workspace è pulito e il branch è corretto.

Prima del fetch:

```
main locale      = commit C
origin/main locale = commit C
```

Sembrerebbe tutto allineato.

Dopo:

```
git fetch origin
```

il quadro diventa:

```
main locale = commit C
origin/main = commit D
```

Ora sappiamo che la precedente impressione di allineamento era falsa. Il **pull --ff-only** porta il locale a **D** e la verifica finale conferma lo stato.

PROC-002 restituisce **PASS**.

18.28 Esempio astratto: working tree modificato

Supponiamo che il controllo iniziale trovi:

```
modified: config/example.yml
```

Il processo non cancella il file, non esegue reset e non assume automaticamente un blocco definitivo.

Applica la disciplina di PROT-001:

```
DIRTY TREE
↓
DIFF / CLASSIFICAZIONE
↓
MODIFICA REALE NON AUTORIZZATA?
├─ SÌ → STOP / BLOCKED
└─ NO → trattamento consentito + verifica
```

Il valore di questo comportamento non è soltanto evitare perdita di lavoro. È conservare **provenance e significato**.

18.29 Esempio astratto: branch divergente

Immaginiamo che il job preveda **main**, ma il workspace sia su **experiment-x**.

Il sistema non deve concludere automaticamente che basti fare checkout di **main**. Prima deve sapere se quel passaggio è compreso nell'authority e se lo stato locale può essere lasciato in sicurezza.

```
BRANCH MISMATCH
↓
AUTHORITY / OWNERSHIP CHECK
↓
CAMBIO AUTORIZZATO E SICURO?
├ SÌ → esegui secondo contratto
└ NO → BLOCKED_WISE
```

Il branch non è un dettaglio di interfaccia: contribuisce all'identità della baseline su cui il lavoro viene eseguito.

18.30 Esempio astratto: fast-forward impossibile

Dopo il fetch emerge:

```
LOCAL:  A → B → C → X
REMOTE: A → B → C → D
```

Le storie sono divergenti. **git pull --ff-only** non può avanzare semplicemente il branch locale.

Questo è esattamente il comportamento desiderato: il comando fallisce **prima** di scegliere arbitrariamente una strategia di fusione.

```
DIVERGENZA RILEVATA
≠
AUTORIZZAZIONE A RISOLVERLA IN QUALSIASI MODO
```

Da qui la causa deve essere classificata e instradata secondo authority e protocolli applicabili.

18.31 Failure mode principali

Il primo failure mode è lavorare prima del fetch, usando come corrente un remote-tracking ref locale stale. Le conseguenze possono essere duplicazione di lavoro, modifica di file già evoluti o valutazione errata della baseline.

Il secondo è usare il sync come licenza a "sistemare Git": qualunque operazione che porta a un tree pulito non è automaticamente accettabile. La post-condizione non può essere raggiunta cancellando arbitrariamente stato o riscrivendo storia.

Il terzo è confondere un problema tecnico con una decisione di governance. Se due linee di lavoro contengono entrambe conoscenza valida, la domanda non è più quale comando Git sblocchi la situazione, ma quale contenuto debba prevalere o essere integrato.

Il quarto è dichiarare **PASS** senza post-check:

```
COMMAND SUCCESS
≠
POST-CONDITION VERIFIED
```

Il branch e il working tree devono essere ancora coerenti con lo stato atteso.

18.32 Che cosa rende questo processo deterministico

PROC-002 non pretende di rendere prevedibile ogni possibile stato Git. Rende prevedibile il **comportamento del sistema davanti a quello stato**.

A parità di condizioni:

```
workspace autorizzato
+
branch corretto
+
working tree sicuro
+
fetch riuscito
+
pull ff-only riuscito
+
post-check positivo
```

l'esito è:

```
PASS
```

Se una preconditione necessaria manca, il sistema non deve interpretare creativamente la situazione per arrivare comunque al lavoro successivo.

Questo è determinismo operativo:

| stesse evidenze strutturali → stesso gate → stessa classe di esito.

18.33 Che cosa PROC-002 non rende deterministico

Esistono situazioni che richiedono interpretazione. Un diff locale può contenere una modifica semanticamente importante. Due branch possono avere evoluzioni entrambe valide. Una divergenza può richiedere una decisione di prodotto, di governance o di

contenuto.

PROC-002 non risolve magicamente queste ambiguità. Il suo compito è impedire che vengano nascoste dentro un'operazione di sincronizzazione.

```
DETERMINISTIC PRE-SYNC
≠
DETERMINISTIC RESOLUTION OF EVERY SEMANTIC CONFLICT
```

Questa distinzione protegge il confine fra Deterministic Core e Cognitive Core.

18.34 Evidence e osservabilità

Un pre-sync utile deve lasciare osservabile ciò che serve a dimostrare l'esito. A seconda dell'implementazione concreta possono essere rilevanti:

- workspace verificato;
- branch osservato;
- stato iniziale del working tree;
- esito del fetch;
- esito del pull fast-forward;
- stato finale;
- eventuale classificazione del blocco.

Il processo canonico non impone qui un unico formato universale di log. Impone però che **PASS** non sia una semplice impressione del runtime: deve essere sostenibile da evidenza operativa sufficiente.

18.35 Maturity: che cosa possiamo affermare oggi

Nel Process Register, PROC-002 è classificato:

```
VALIDATED
```

La baseline tecnica indica che la necessità del fetch esplicito e del pull fast-forward è emersa nei POC del Service Bridge ed è stata successivamente applicata con successo nei run zero-touch e routine-driven.

Questo permette di considerare il processo parte della baseline validata corrente.

Non significa però che ogni possibile variante di repository, provider Git, topologia enterprise, policy security o scenario di conflitto sia stata validata universalmente.

La formulazione corretta è quindi:

PROC-002 è un processo WCM validato nella baseline corrente; l'applicazione a contesti ulteriori resta soggetta ai vincoli e alle caratteristiche del contesto operativo.

18.36 Source map del capitolo

Questo capitolo deriva principalmente da:

```
wcm/process-book/processes/PROC-002_WORKSPACE_PRE_SYNC.md
wcm/process-book/protocols/PROT-001_GIT_WORKTREE_SAFETY.md
wcm/process-book/protocols/PROT-006_BRANCH_OWNERSHIP_BASELINE_SYNC.md
wcm/process-book/PROCESS_REGISTER.md
WCM_AGENT_START.md
```

Il rapporto tra le fonti è:

```
PROC-002
→ obiettivo, trigger, sequenza, regole ed esiti del pre-sync

PROT-001
→ guardrail di sicurezza Git e working tree

PROT-006
→ disciplina corrente di trunk, branch temporanee e baseline sync

PROCESS_REGISTER
→ identità, stato e collocazione del processo nella baseline

WCM_AGENT_START
→ invarianti generali di authority e fail-closed
```

Il capitolo non introduce nuovi requisiti tecnici oltre queste fonti.

18.37 La scheda di lettura di PROC-002

Campo	Lettura
ID	PROC-002
Nome	Workspace Pre-Sync
Stato	VALIDATED
Owner	Execution Service
Scopo	Rendere affidabile un workspace Git locale rispetto alla baseline remota prima dell'esecuzione
Trigger	Lavoro che dipende dallo stato corrente di un repository Git remoto
Input	workspace autorizzato, branch atteso, working tree, remote origin, authority applicabile
Flusso	verify workspace → verify branch → verify tree → fetch → pull ff-only → post-check
Output	workspace locale affidabile oppure blocco classificato

Gate principali	workspace identity, branch, working tree safety, remote freshness, fast-forward compatibility, post-condition
Esiti	PASS / BLOCKED_LOCAL / BLOCKED_WISE
Protocolli principali	PROT-001; PROT-006 come disciplina di branch/baseline
Relazioni	abilita lavoro successivo del Service Job quando la baseline locale deve essere affidabile
Failure mode	stato stale, branch errato, dirty tree non compreso, sync distruttivo, divergenza forzata, PASS senza post-check
Maturity	VALIDATED nella baseline corrente

18.38 Il significato sistemico di PROC-002

PROC-002 può sembrare uno dei processi più tecnici del WCM. In realtà esprime una regola organizzativa molto più generale:

prima di prendere una decisione o compiere un'azione su uno stato condiviso, devi dimostrare che la copia su cui stai lavorando rappresenta davvero la baseline che credi di usare.

Nel caso di PROC-002 questa regola assume una forma concreta attraverso Git. Ma il principio sottostante riguarda freschezza, provenance, authority, post-condition verification, protezione dal drift e separazione fra osservazione e mutazione.

Per questo il Workspace Pre-Sync non è un dettaglio amministrativo prima del "vero lavoro". È parte del vero lavoro.

18.39 Da PROC-002 a PROC-003

Ora il Service Job può avere un lifecycle persistente e, quando l'esecuzione dipende da un workspace locale, possiamo verificare che quel workspace rappresenti una baseline affidabile.

Rimane però un'altra domanda:

Come facciamo a scoprire lavoro eleggibile e ad attivarlo senza usare continuamente un LLM e senza generare dispatch duplicati?

È il problema del prossimo processo:

PROC-003
DETERMINISTIC DISCOVERY
& DURABLE DISPATCH

Nel prossimo capitolo passeremo quindi dalla preparazione sicura del workspace alla meccanica con cui il sistema scopre, deduplica e consegna lavoro eseguibile.